製造指示一覧 — CSV出力 — 機能設計

> **2026-10-24 改版（WI-017）**。本ファイルは `002_DD-FN-CSV_製造指示一覧.md` を置き換える。旧ファイルは承認済みの前版として変更しない。変更内容：製造指示の備考の上限を1000文字とする（REQ-090。REQ-016 の500文字を置き換える）。詳細は `work-items/WI-017/` を参照。

002_DD-FN-CSV — 002_DD-CSV と 002_DD-API-CSV（API-PO-05）が使用する。002_BD-CSV の FN-041 と FN-042 を実装する。要件 REQ-085〜REQ-088。

改版履歴

項目	値
文書ID	002_DD-FN-CSV
システム名	ProductionManagementAI
サブシステム名	製造指示
作業項目	WI-016
作成者	Claude (for ThongTM)
作成日	2026-10-07
最終更新者	Claude (for ThongTM)
最終更新日	2026-10-24

版	日付	作成者	改版内容
1	2026-10-07	Claude (for ThongTM)	新規作成
2	2026-10-24	Claude (for ThongTM)	WI-017（REQ-090〜REQ-092）：備考の上限を500文字から1000文字へ変更。出力行の `Notes` の長さを1000以内とした。処理の変更なし。ファイル名に日付を付与（WI-017 DEC-002）

概要とメソッド一覧

項目	値
モジュール名	`ProductionOrderService`（出力）、`IProductionOrderRepository`（出力クエリ）、`ProductionOrderCsvWriter`、`CsvExportResult`、`IPlantClock`（1つ追加）
概要	API-PO-05 のサーバー側：チェックし、読み取りスナップショット内で件数を数え、10,000件を超えれば拒否し、行を安全な CSV としてストリーミングし、出力をログとトレースに記録する

No	メソッド名	概要	備考
1	`ProductionOrderService.ExportAsync`	製品の確認、スナップショット、件数、件数上限、テレメトリー。開いた出力またはエラーの結果を返す	アプリケーション、新規
2	`IProductionOrderRepository.BeginReadSnapshotAsync` / `StreamExportRowsAsync`	読み取り専用の REPEATABLE READ トランザクション。絞り込み・並べ替え済み、ページングなしの行ストリーム	ポート＋インフラストラクチャ、新規。`CountOrdersAsync` は再利用
3	`ProductionOrderCsvWriter.WriteAsync` / `FormatRow` / `Field`	BOM、ヘッダー行、行の書式、RFC 4180 の引用符付け、数式の無害化	アプリケーション、新規。出力ストリーム以外は純粋
4	`CsvExportResult.ExecuteResultAsync`	API-PO-05 のヘッダーを設定し、ライターを実行し、ストリーミング中の失敗では接続を中断し、スナップショットを破棄する	Api、新規
5	`IPlantClock.ToPlantTime`	UTC の瞬間 → 工場ローカルの `DateTime`	ポート＋ `PlantClock`、追加

共通ユーティリティ参照

No	名前	概要	備考
1	`ProductionOrderListQuery.TryCreate`	一覧のチェック関数。出力は `Page` と `PageSize` を null にした `ProductionOrderListRequest` を作る	変更なし
2	`ProductionOrderQueryExtensions.ApplyFilters` / `ApplySort`	一覧の条件と並び順キーの対応（同順位は指示番号）	変更なし。共有することで出力が画面と食い違うことはない
3	`ProductionOrderProblems.ToActionResult`	`Result` → Problem Details（400/422/…）	変更なし
4	`ProductionOrderTelemetry`	アクティビティソース、メーター、結果名	計器を2つ追加（メソッド1）
5	`ProductionOrderMessages`（`Msg`）	メッセージID の定数	`ExportLimitExceeded = "MSG-E024"` を追加

リクエストデータ

なし — 本モジュールは他の API やサービスを呼ばない。リポジトリのポート経由でデータベースを読む。

レスポンス／値の変換

CSV の列と値の変換は 002_DD-API-CSV（「レスポンス本文（CSV）」）と 002_BD-CSV M-11〜M-15 で一度だけ定義し、メソッド3 が実装する。共通の行の型：

No	名前	変数名	型	長さ	必須	値の変換	備考
1	指示番号	`OrderNumber`	string	13	yes	—	
2	製品 SKU／名称／単位	`ProductSku`、`ProductName`、`ProductUnit`	string	50 / 200 / —	yes	—	
3	ライン	`Line`	`OrderLineResponse?`	—	no	M-14	一覧の行と同じ射影
4	数量	`Quantity`	decimal	—	yes	M-12	
5	納期	`DueDate`	`DateOnly`	—	yes	M-13	
6	ステータス	`Status`	`ProductionOrderStatus`	—	yes	M-11	
7	備考	`Notes`	string?	≤ 1000	no	—	一覧の行にはない。出力のために追加（DEC-003）
8	作成／更新／完了	`CreatedAtUtc`、`UpdatedAtUtc`、`CompletedAtUtc`	`DateTimeOffset` / `DateTimeOffset` / `DateTimeOffset?`	—	—	M-13	

`ProductionOrderExportRow` は `ProductionOrderListRow` と並ぶ新しいレコード。一覧の行は広げないため、一覧の射影（002_DB）は変わらない。

メソッド設計

1. `ProductionOrderService.ExportAsync`

項目	値
説明	チェック済みのクエリについて出力を準備する：製品フィルターを確認し、読み取りスナップショットを開き、件数を数え、件数上限を適用し、開いた出力（件数＋行ストリーム＋工場時刻）または `Result` のエラーを返す
戻り値の型	`Task<Result<ProductionOrderExport>>`
作成者／日付	Claude / 2026-10-07
最終更新者／日付	—

引数

No	型	名前	説明
1	`ProductionOrderListQuery`	`query`	`TryCreate` でチェック済み（ページングの項目は既定値で未使用）
2	`string?`	`userId`	コントローラーから渡す呼び出し元のユーザーID（ `ClaimTypes.NameIdentifier` ）。ログのみに使用
3	`CancellationToken`	`cancellationToken`	リクエストの中断

戻り値

型	名前	説明
`Result<ProductionOrderExport>.Ok`	export	`int Count` 、`DateTime PlantNow` 、`DateOnly PlantToday` 、`IAsyncEnumerable<ProductionOrderExportRow> Rows` と所有するスナップショットを持つ `ProductionOrderExport : IAsyncDisposable` 。呼び出し元（メソッド4）が破棄すること
`Result<…>.Invalid`	—	`productId: MSG-E002` （一覧と同じ）
`Result<…>.RuleViolation`	—	`MSG-E024` 、件数 > `ProductionOrderExport.MaxRows` （10,000、DEC-005）

処理概要： `ListAsync` と同じ構成（同じアクティビティのタグ、同じ製品の確認）だが、ページングなしで、読み取り専用の REPEATABLE READ トランザクション内で実行する。そのため、他のユーザーが製造指示を保存している間も、 `Count` （ `X-Total-Count` として送る）とストリーミングする行は1つのスナップショットから得られる。件数上限のチェックを通過するまで行は読まない。スナップショットは Ok 以外のすべての経路ではここで、Ok の経路ではメソッド4 で破棄する。

処理フロー

手順	説明	呼び出し
1	アクティビティ `ProductionOrder.Export` を開始。一覧のクエリタグを設定（ `SetQueryTags` 、ページのタグは除く）	`ProductionOrderTelemetry.Source`
2	製品フィルターがあり存在しない → `Invalid { productId: [MSG-E002] }` 、結果 `validation_failed`	`ProductExistsAsync` （既存）
3	スナップショットを開く	メソッド2 `BeginReadSnapshotAsync`
4	`count = CountOrdersAsync(query)`	既存
5	`count > 10,000` → スナップショットを破棄、`RuleViolation(MSG-E024)` 、結果 `rule_violation` 、結果 `limit` でログ 2005	—
6	`PlantNow = ToPlantTime(utcNow)` と `PlantToday` を1回だけ読み、ファイル名、納期遅れの列、すべての行を一致させる	メソッド5
7	`Ok(new ProductionOrderExport(count, PlantNow, PlantToday, StreamExportRowsAsync(query), snapshot))` を返す	メソッド2
8	手順7 より前の予期しない例外：スナップショットを破棄、結果 `error` 、`LogUnexpectedFailure(…, "export")` 、再スロー（→ 500 `MSG-E013` ）	—

テレメトリー（ `ProductionOrderTelemetry` への追加）：

計器	名前	タグ	記録
カウンター	`pmai.production_orders.exported`	`outcome` = `success` | `validation_failed` | `rule_violation` | `error` | `cancelled`	出力ごとに1回：ここでは手順2/5/8、success/error/cancelled はメソッド4
ヒストグラム	`pmai.production_orders.export_rows`	—	成功時に書き込んだ行数（メソッド4）
アクティビティ	`ProductionOrder.Export`	一覧のフィルター／並び順のタグ、`result.total` 、`outcome`	サービス呼び出しとストリーミングにまたがる（メソッド4 で終了）
ログ	EventId 2005 `ProductionOrdersExported` 、Information： `Production orders export by {UserId}: {Outcome}, {RowCount} rows in {DurationMs} ms`	—	出力ごとに1件。製造指示の内容、備考、フィルターの値はログに出さない（002_BD-CSV の可観測性）

2. `IProductionOrderRepository.BeginReadSnapshotAsync` と `StreamExportRowsAsync`

項目	値
説明	`BeginReadSnapshotAsync(ct) → IProductionOrderTransaction` ：コンテキストの接続で `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY` 。 `StreamExportRowsAsync(query) → IAsyncEnumerable<ProductionOrderExportRow>` ： `Skip` / `Take` なしの一覧クエリで、出力行を射影する
戻り値の型	`Task<IProductionOrderTransaction>` 、`IAsyncEnumerable<ProductionOrderExportRow>`
作成者／日付	Claude / 2026-10-07

項目	値
最終更新者／日付	—

引数： `ProductionOrderListQuery query`、`CancellationToken cancellationToken`（`WithCancellation` 経由）。

戻り値： 説明を参照。`CountOrdersAsync`（既存）は、どちらもスコープ付きの `DbContext` の接続を使うため、同じトランザクション内で実行される。

処理概要：`db.ProductionOrders.AsNoTracking().ApplyFilters(query).Join(products).ApplySort(query.Sort, query.Direction).Select(new ProductionOrderExportRow(…, line subquery as the list, Notes, CreatedAtUtc, UpdatedAtUtc, CompletedAtUtc)).AsAsyncEnumerable()`。EF Core はデータリーダーでストリーミングするため、一度に実体化するのは最大1行（`ToListAsync` なし）。クエリは 002_DB と WI-004 のフィルター・並び順のインデックスを使う。なくなるのは `LIMIT/OFFSET` のみで、行数はメソッド1 の手順5 で上限が決まる。`IsolationLevel.RepeatableRead` と `READ ONLY` は PostgreSQL 標準（`pmai_app` にはどちらも追加の権限が不要）。制限付きのアプリのログインはこれらのテーブルに `SELECT` のみを持ち続け、権限の変更はない。

処理フロー

手順	説明	呼び出し
1	`RepeatableRead` のトランザクションを開始し、`SET TRANSACTION READ ONLY`	`db.Database.BeginTransactionAsync(IsolationLevel.RepeatableRead)`
2	クエリを作成：`ApplyFilters` → `products` と結合 → `ApplySort` → 射影	`ProductionOrderQueryExtensions`（変更なし）
3	`AsAsyncEnumerable()` を返す — メソッド3 が列挙するときに実行される	—

3. `ProductionOrderCsvWriter`

項目	値
説明	API-PO-05 の本文を書く、カルチャーに依存しない static のライター。`WriteAsync(export, Stream output, ct)` は BOM、ヘッダー行、すべての行を書く。`FormatRow(row, plantToday, toPlantTime)` と `Field(string?, bool isText)` は単体テストのため public
戻り値の型	`Task<int>`（書き込んだ行数）、`string`、`string`
作成者／日付	Claude / 2026-10-07
最終更新者／日付	—

引数

No	型	名前	説明
1	`ProductionOrderExport`	`export`	メソッド1 から

No	型	名前	説明
2	`Stream`	`output`	`HttpResponse.Body`（メソッド4）。単体テストでは `MemoryStream`
3	`CancellationToken`	`cancellationToken`	`HttpContext.RequestAborted`

戻り値

型	名前	説明
`int`	rows	書き込んだ行数。どちらも1つのスナップショットから得るため `export.Count` と一致する。不一致は（スナップショットの不具合を示すため）隠さずに警告としてログに出す

処理概要と正確なルール：

- **文字コード：** `EF BB BF` を明示的に書き、続いて `new UTF8Encoding(false)`（2つ目の BOM なし）、`NewLine = "\r\n"` の `StreamWriter` で、非同期の書き込みのみ（Kestrel は同期 I/O を許可しない）。クエリの実行中もブラウザーがデータを受け取れるよう500行ごとにフラッシュし、最後に `FlushAsync`。
- **ヘッダー行：** 002_DD-API-CSV の13個の日本語の見出しを順に。
- **値**（M-11〜M-15）：ステータスの表示は固定の `switch`（`下書き`、`進行中`、`完了`、`取消`）。数量は `ToString("0.###", CultureInfo.InvariantCulture)`（小数3桁以内、桁区切りなし、末尾の0なし）。納期は `yyyy/MM/dd`。日時は `toPlantTime(utc).ToString("yyyy/MM/dd HH:mm", InvariantCulture)`、null なら空。納期遅れは `DueDate < plantToday && Status is Draft or InProgress` のとき `納期遅れ`（`ProductionOrderListMapper.ToListItem` と同じ条件で、2つがずれないよう共通の `IsOverdue` ヘルパーに抽出）。ラインは `{code} – {name}`、`!IsActive` なら `(使用停止)` を付加、null なら空。
- **`Field(value, isText)`：** null → 空。`isText` で値が `=`、`+`、`-`、`@`、`\t`、`\r` で始まる場合は `'` を前置する（OWASP CSV インジェクション、CWE-1236）。次に `,`、`"`、`\r`、`\n` を含む場合は `"` で囲み、各 `"` を二重にする（RFC 4180）。文字列の列：1、2、3、4、6、8、9、10。数値・日付の列（5、7、11〜13）は上記の書式関数が作り、前置しない。備考内の改行は保存値のまま保持する（DEC-013）。

処理フロー

手順	説明	呼び出し
1	BOM とヘッダー行を書く	—
2	`export.Rows.WithCancellation(ct)` の各行を `await foreach`：`FormatRow(…)` ＋ CRLF を書く。500行ごとに `FlushAsync`	メソッド2 のストリーム
3	最後に `FlushAsync`、行数を返す	—

4. `CsvExportResult.ExecuteResultAsync`

項目	値
説明	`Ok` のとき `ProductionOrdersController.Export` が返す `IActionResult`。レスポンスが終わるまで `ProductionOrderExport` を所有する
戻り値の型	`Task`
作成者／日付	Claude / 2026-10-07
最終更新者／日付	—

引数： `ActionContext context`（フレームワーク）。出力、ロガー、開始時刻で生成する。

処理概要：002_DD-API-CSV の 200 のヘッダーを設定する — `Content-Type`、`Content-Disposition`（`ContentDispositionHeaderValue` で作成、`FileName` = `production-orders_{PlantNow:yyyyMMdd-HHmm}.csv`、`FileNameStar` = 製造指示一覧_{…}.csv）、`X-Total-Count = export.Count` — その後 `Response.Body` に対してメソッド3 を呼ぶ。最初のバイトより前のエラーはここでは起こらない（すべてのチェックはメソッド1 で実行済み）。ヘッダー送信後に例外が発生すると、レスポンスはもう Problem Details にできないため `HttpContext.Abort()` を呼ぶ：ブラウザーはネットワークエラーとして受け取り、002_DD-CSV は MSG-E025 を表示する。短いファイルが成功として報告されることはない。`finally` で出力を破棄し（スナップショットのトランザクションを終了）、カウンター・ヒストグラムを記録し、アクティビティを終了し、ログ 2005 を書く。

処理フロー

手順	説明	呼び出し
1	ステータス 200 とヘッダーを設定	—
2	`rows = await ProductionOrderCsvWriter.WriteAsync(export, Response.Body, RequestAborted)`	メソッド3
3	成功 → カウンター `success`、ヒストグラム `rows`、ログ 2005 `success`	メソッド1 のテレメトリー
4	`RequestAborted` が設定された `OperationCanceledException` → 結果 `cancelled`（ユーザーが離れた、または表示が変わった）。他に何もしない	—
5	その他の例外 → ログ `LogUnexpectedFailure(…, "export")`、結果 `error`、`HttpContext.Abort()`	—
6	`finally`：`await export.DisposeAsync()`	—

5. `IPlantClock.ToPlantTime`

項目	値
説明	`DateTime ToPlantTime(DateTimeOffset utc)`：瞬間の工場ローカルの時刻（`TimeZoneInfo.ConvertTime(utc, zone).DateTime`）。日時の列とファイル名に使う（DEC-008）
戻り値の型	`DateTime`（Kind `Unspecified`）

項目	値
作成者／日付	Claude / 2026-10-07
最終更新者／日付	—

ポートと `PlantClock` への追加。`DateOf` の動作は変わらず、これに委譲できる。`tests/` の `IPlantClock` のテストダブルにこのメンバーを1つ追加する。

未決事項

なし。承認済みの設計の範囲内で本書で選択したもの：件数と行のための REPEATABLE READ READ ONLY スナップショット、一覧の行とは別の `ProductionOrderExportRow`、共通の `IsOverdue` ヘルパー、500行ごとのフラッシュ、ログの EventId 2005、上記のメトリクス名。本書の提示時に WI-016 DEC-015 として記録。